

Итоговый отчет о выполненной работе

Исследование стоимости сопряжений на циклических кривых в EVM и перспектив дешевого folding

MNT4-753, MNT6-753 и исследовательская конструкция lollipop-305

Содержание

1	Постановка задачи и практическая мотивация	2
1.1	Статус результата	3
2	Теоретическая база вычисления сопряжения	4
2.1	Цикл Миллера и финальная экспонента	4
2.2	Удаление знаменателя	4
2.3	Проверка финальной экспоненты через свидетельство	5
3	Оптимизированная 3-limb арифметика MNT4-753	6
3.1	Представление данных	6
3.2	Почему выбран Montgomery/CIOS	6
3.3	Нижняя оценка для 3-limb умножения	7
3.4	Башня расширений и Karatsuba	9
3.5	Подготовленные разреженные линии	9
3.6	Низкоуровневые оптимизации EVM	11
4	MNT4: от наивного Tate pairing к оптимизированному пути	11
4.1	Наивная точка отсчета	11
4.2	Оптимизационная лестница	12
4.3	Формальная нижняя оценка стоимости Miller core	13
5	MNT4 direct residue verifier по ePrint 2024/640	14
5.1	API и граница доверия	14
5.2	Результаты MNT4	14
6	Проверка цикла Миллера через полиномиальные отношения	15
6.1	Математическая идея	15
6.2	Построенная MNT4-модель	15
6.3	KZG over BN254	16
6.4	Merkle/FRI	17
7	MNT6-753 и цикл MNT4/MNT6	18
7.1	Почему нужна вторая кривая	18
7.2	MNT6 arithmetic и residue verifier	19
7.3	Cycle-native accounting	20

8	Исследовательское направление lollipop-305	22
8.1	Идея	22
8.2	Кривые конструкции	22
8.3	Оптимизированная 2-limb арифметика	23
8.4	Почему для третьей кривой нужны два аккумулятора	24
8.5	Результаты lollipop-305	25
9	Сводные результаты и trade-off	25
9.1	Три формы стоимости	26
9.2	Экономическая интерпретация	26
10	Итоговые выводы	27
10.1	Закрытие замечаний научного руководителя	27
11	Воспроизводимость	29

1 Постановка задачи и практическая мотивация

Работа посвящена проверяемым вычислениям на эллиптических кривых в EVM. Практическая мотивация связана с рекурсивными доказательствами. Если требуется построить цепочку доказательств, недостаточно один раз проверить отдельное утверждение. Нужно уметь эффективно доказывать корректность предыдущего шага внутри следующего шага. На этой идее строятся folding-схемы и рекурсивные SNARK-системы.

В существующих реализациях, включая Sonobe/CycleFold, заметная часть стоимости может возникать не из-за пользовательского утверждения, а из-за переноса арифметики одной кривой в поле другой кривой. Такая арифметика называется *non-native*. Элемент большого поля раскладывается на машинные слова (*limb*-ы), а каждое умножение требует ограничений на частичные произведения, переносы и редукцию. В качестве практического ориентира в работе используется порядок около 9 млн constraints для Sonobe/CycleFold-like decider. Этот показатель не является прямым apples-to-apples сравнением с каждым отдельным фрагментом настоящей работы, но хорошо показывает масштаб проблемы.

Для Ethereum задача дополнительно осложняется архитектурой EVM:

- EVM работает с 256-битными словами;
- для BN254 существуют предкомпилированные операции;
- для MNT4-753 и MNT6-753 аналогичных предкомпилированных операций нет;
- элементы MNT-полей имеют размер около 753 бит и требуют трех слов EVM.

Поэтому были исследованы два фундаментальных пути:

1. выполнять арифметику сопряжения непосредственно on-chain и платить за нее gas;
2. переносить вычисление off-chain и платить за доказательство, constraints, calldata и время работы доказчика.

Цель работы состоит не в том, чтобы заранее выбрать один путь, а в том, чтобы реализовать и измерить основные варианты, определить границу применимости и проверить, дает ли цикл MNT4/MNT6 или конструкция lollipop более дешевую основу для будущего folding.

1.1 Статус результата

Практическая исследовательская часть завершена как всестороннее исследование границы применимости. Работа не ограничилась написанием одного контракта: последовательно исследованы математические преобразования, алгоритмы арифметики, низкоуровневая реализация в EVM, прозрачные и KZG-подходы к проверке трассы, цикл MNT4/MNT6 и альтернативное семейство кривых lollipop-305.

Получены следующие теоретические результаты:

- формализована корректность удаления вертикальных знаменателей в функции Миллера для выбранной MNT4-постановки;
- отдельно формализована допустимость использования ненормализованных `projective/prepared` коэффициентов линий;
- выведено отношение со свидетельством c , которое заменяет длинную финальную экспоненту при проверке уравнения сопряжений;
- для MNT6 выведен отдельный знак степени: $r_{\text{MNT6}} = q_{\text{MNT6}} - N$, поэтому проверяется $F \cdot c^{N-q} = 1$;
- построена формальная модель проверки цикла Миллера через полиномиальные отношения, KZG-opening и Merkle/ordinary-FRI;
- доказана корректность отношений полей цикла MNT4/MNT6;
- для lollipop-305 исправлена башня расширения, проверены twist- и Frobenius-отношения и показано, почему для третьей кривой требуется отдельная работа со знаменателями.

Получены следующие инженерные результаты:

- реализована оптимизированная 3-limb арифметика MNT4-753 и MNT6-753 в Solidity/Yul, включая поля расширений;
- реализованы и измерены Montgomery/CIOS, Barrett, FIOS, Comba/SOS, lazy-reduction и branchless-варианты;
- построены наивная Tate-модель, полное on-chain MNT4-сопряжение, prepared fixed- Q путь и Article640 residue verifier;
- реализован полный MNT6 Article640 verifier с общим multi-Miller аккумулятором;
- реализованы KZG-opening over BN254, Merkle-opening слой, исполняемый отрицательный DEEP-FRI эксперимент и ordinary-FRI cost model;
- реализована 2-limb арифметика и три pairing-компонента исследовательской конструкции lollipop-305;
- добавлены cross-check-и против arkworks/Rust reference и воспроизводимые gas-бенчмарки.

Главный результат имеет граничный характер. Низкоуровневые оптимизации сокращают стоимость на порядок, но без новых системных возможностей нельзя одновременно получить малую стоимость on-chain и малую стоимость off-chain proof-слоя простым переносом арифметики между слоями. При этом MNT4/MNT6 cycle-native accounting показывает перспективное продолжение: арифметику можно формулировать в родных полях цикла и тем самым избежать наиболее дорогой BN254 non-native эмуляции.

2 Теоретическая база вычисления сопряжения

2.1 Цикл Миллера и финальная экспонента

Для reduced Tate pairing используется вычисление функции Миллера и финальная экспонента:

$$e(P, Q) = f_{r,P}(Q)^{(q^k-1)/r}.$$

Здесь r — порядок подгруппы, q — характеристика базового поля, а k — степень вложения. Алгоритм Миллера строит $f_{r,P}$ через последовательность удвоений и сложений. Для каждого шага обновляется аккумулятор:

$$f \leftarrow f^2 \cdot \ell_{\text{dbl}}(Q),$$

а для ненулевой цифры signed-представления дополнительно:

$$f \leftarrow f \cdot \ell_{\text{add}}(Q).$$

Для уравнения сопряжений

$$e(P, Q) = e(R, S)$$

удобно проверять эквивалентную форму

$$e(P, Q) \cdot e(-R, S) = 1.$$

Обе пары можно обрабатывать общим аккумулятором:

$$f \leftarrow f^2 \cdot \ell_{Q,i}(P) \cdot \ell_{S,i}(-R).$$

Главное преимущество общего аккумулятора: возведение в квадрат выполняется один раз на раунд, а не отдельно для каждой пары.

2.2 Удаление знаменателя

В общей формуле функции Миллера возникает отношение линии к вертикальной линии:

$$g_{A,B}(X) = \frac{\ell_{A,B}(X)}{v_{A+B}(X)}.$$

Для MNT4-753 знаменатель после выбора групп лежит в собственном подполе. Финальная экспонента уничтожает этот множитель. Поэтому в вычислительном пути можно использовать только числитель линии. В полном тексте диплома приведено формальное доказательство корректности такого удаления.

Практический смысл оптимизации существенен: на каждом шаге не требуется вычислять дополнительное обращение или вести второй аккумулятор знаменателя. Это свойство не является автоматически верным для любой кривой. Ниже будет показано, что для третьей кривой lollipop-305 оно не выполняется в нужной форме, и это напрямую увеличивает стоимость.

Два разных уровня удаления знаменателей. В работе используются две связанные, но не тождественные оптимизации.

Первая оптимизация относится к самой функции Миллера. В точной формуле нужно умножать аккумулятор на

$$\frac{\ell_{A,B}(X)}{v_{A+B}(X)}.$$

Если вертикальная линия $v_{A+B}(X)$ после подстановки точки лежит в собственном подполе, ее вклад уничтожается финальной экспонентой. Поэтому динамический цикл может не накапливать знаменатель.

Вторая оптимизация относится к построению подготовленного кэша линий. При вычислении линии в проективных координатах ее коэффициенты определены с точностью до ненулевого масштабирующего множителя:

$$(\lambda_0, \lambda_1, \lambda_2) \sim (s\lambda_0, s\lambda_1, s\lambda_2).$$

Приведение коэффициентов к аффинной нормальной форме потребовало бы обращений в поле. В fixed-cache пути хранятся projective/prepared коэффициенты без этой нормализации. Масштабирующие множители допустимы только при выполнении предположек конкретного pairing-equation пути: они либо уничтожаются финальной экспонентой, либо учитываются при построении зафиксированного кэша. Таким образом, первая оптимизация удаляет вертикальные знаменатели Miller-функции, а вторая устраняет дорогое нормализующее деление при подготовке line-cache.

2.3 Проверка финальной экспоненты через свидетельство

Статья *On Proving Pairings* [1] позволяет заменить полную финальную экспоненту проверкой отношения. Для уравнения сопряжений пусть F — результат объединенного цикла Миллера. Требуется проверить:

$$F^{(q^k-1)/r} = 1.$$

При выполнении условий теоремы статьи это равносильно существованию элемента c , для которого:

$$F = c^r.$$

Вместо длинной цепочки возведений в степень контракт проверяет эквивалентное отношение:

$$F \cdot c^{-r} = 1.$$

Часть степени c^{-r} встраивается в тот же signed double-and-add цикл, а возведение в степень q выполняется отображением Фробениуса.

Для MNT4 и MNT6 знаки различаются:

$$r_{\text{MNT4}} = q_{\text{MNT4}} + N, \quad r_{\text{MNT6}} = q_{\text{MNT6}} - N.$$

Следовательно:

$$c^{-r_{\text{MNT4}}} = c^{-N-q}, \quad c^{-r_{\text{MNT6}}} = c^{N-q}.$$

Это различие было учтено в финальной реализации MNT6.

3 Оптимизированная 3-limb арифметика MNT4-753

3.1 Представление данных

Элемент базового поля MNT4-753 не помещается в одно слово EVM и хранится как три 256-битных слова:

$$x = x_0 + 2^{256}x_1 + 2^{512}x_2.$$

Для длинных цепочек умножений используется Montgomery-представление. Пусть

$$B = 2^{256}, \quad R = B^3 = 2^{768}.$$

Тогда:

$$\tilde{x} = xR \bmod p,$$

$$\text{MontMul}(\tilde{x}, \tilde{y}) = \tilde{x}\tilde{y}R^{-1} \bmod p = xyR \bmod p.$$

Результат остается в Montgomery-домене. Это важно для цикла Миллера: вход в домен выполняется один раз, после чего тысячи операций не требуют повторного преобразования.

3.2 Почему выбран Montgomery/CIOS

Были реализованы и измерены несколько вариантов:

- Montgomery/CIOS;
- Barrett-редукция;
- FIOS;
- Comba/SOS;
- square-specialized Comba;
- branchless reduction;
- lazy reduction variants.

CIOS (*Coarsely Integrated Operand Scanning*) объединяет накопление частичных произведений и Montgomery-редукцию. Для EVM это выгоднее, чем материализация полного 6-словного произведения в памяти.

Таблица 1: Сравнение базовых 3-limb операций

Вариант	Операция	Gas/op	Вывод
Montgomery/CIOS	F_q mul	2,959	основной путь
Montgomery/CIOS	F_q square	2,947	основной путь
Barrett	F_q mul	46,998	существенно дороже
Barrett	F_q square	47,172	существенно дороже
FIOS	F_q mul	18,509	дороже CIOS
FIOS	F_q square	18,543	дороже CIOS
Comba/SOS	F_q mul	18,301	проигрывает из-за memory traffic
Square-Comba	F_q square	18,420	лучше общего Comba, но хуже CIOS

3.3 Нижняя оценка для 3-limb умножения

Для произведения двух трехсловных чисел требуется не менее девяти попарных произведений:

$$ab = \sum_{i=0}^2 \sum_{j=0}^2 a_i b_j 2^{256(i+j)}.$$

Montgomery-редукция выполняет три шага. На каждом шаге добавляется произведение корректирующего коэффициента на трехсловный модуль, то есть еще три произведения на шаг. Получаем:

$$9 \text{ произведений для } ab + 9 \text{ произведений для REDC} = 18$$

полных 512-битных limb-произведений.

В EVM нет инструкции MULHI. Для восстановления старшей половины 512-битного произведения используется связка:

```
lo := mul(u, v)
mm := mulmod(u, v, not(0))
hi := sub(sub(mm, lo), lt(mm, lo))
```

При действующем расписании EVM опкоды имеют стоимость:

$$\text{MUL} = 5, \quad \text{MULMOD} = 8, \quad \text{ADD} = \text{SUB} = \text{LT} = 3.$$

Следовательно, одно восстановление полного 512-битного произведения требует как минимум:

$$5 + 8 + 3 + 3 + 3 = 22 \text{ gas}.$$

Только неизбежные 18 произведений дают:

$$18 \cdot 22 = 396 \text{ gas}.$$

Это строгая нижняя граница только для восстановления произведений. Ее нельзя сравнивать с измеренным значением как с оценкой полной функции: в ней еще нет накопления частичных сумм, вычисления коэффициентов CIOS, нормализации и служебных инструкций стековой машины.

Получим более содержательную нижнюю границу для арифметического каркаса функции. Каждый 512-битный результат имеет младшую и старшую половины. Даже если предположить идеальное расписание, обе половины необходимо добавить в многословный аккумулятор и проверить перенос:

$$18 \cdot 2 \cdot (\text{ADD} + \text{LT}) = 18 \cdot 2 \cdot (3 + 3) = 216 \text{ gas.}$$

Три CIOS-шага дополнительно вычисляют младший корректирующий коэффициент:

$$m_i = t_0 \cdot (-q^{-1}) \bmod 2^{256},$$

то есть требуют еще трех инструкций MUL. Уже до условного финального вычитания модуля получаем:

$$396 + 216 + 3 \cdot 5 = 627 \text{ gas.}$$

Граница 627 gas остается намеренно заниженной. В реальном accumulator-е нельзя ограничиться одной проверкой переноса на каждую половину: перенос может последовательно затрагивать следующие слова. Кроме того, необходимо сдвигать окно CIOS после каждой итерации и в конце условно вычесть модуль.

Для проверки масштаба накладных расходов отдельно разобран фактически исполняемый all-stack Yul-код. В нем полностью развернуты три CIOS-итерации: циклов по limb-ам и промежуточных массивов в памяти нет. После раскрытия вспомогательных функций структура кода содержит операции из табл. 2.

Таблица 2: Структурный opcode-состав all-stack montMul3

Группа	Число	Gas/opcode	Сумма, gas
MUL для limb-произведений	18	5	90
MULMOD для старших половин	18	8	144
SUB и LT внутри mul512	36 + 18	3	162
MUL для коэффициентов CIOS	3	5	15
ADD накопления и переносов	до 83	3	до 249
LT сети переносов и финального сравнения	до 48	3	до 144
JUMPI/JUMPDEST условных ветвей	до 23	10 + 1	до 253

Таблица не является точным трассировочным отчетом для одного набора входов: часть ветвей выполняется только при возникновении переноса, а финальное вычитание зависит от результата. Но она показывает, почему сравнение 2,959 с числом 627 вводило бы в заблуждение. После неизбежных произведений значимую стоимость имеют сеть переносов и управление ветвями. Дополнительно EVM исполняет загрузку констант, PUSH, DUP, SWAP, финальную нормализацию и служебный код, который не отражен в арифметическом каркасе.

Следовательно, корректно фиксировать два разных результата:

- 627 gas — строгая, но заведомо неполная нижняя граница арифметического кар-каса;
- 2,959 gas/op — измеренная стоимость лучшего реализованного внутреннего all-stack Montgomery/CIOS пути.

Глобальная оптимальность среди всех возможных EVM-bytecode программ не утверждается. Практическая обоснованность выбора подтверждается сравнением с реализованными альтернативами: Barrett, FIOS, Comba/SOS, branchless reduction и вариантами финального вычитания. Ни одна из них не снизила стоимость минимум на 10%.

3.4 Башня расширений и Karatsuba

Для MNT4 используется башня:

$$\mathbb{F}_{q^2} = \mathbb{F}_q[u]/(u^2 - 13), \quad \mathbb{F}_{q^4} = \mathbb{F}_{q^2}[v]/(v^2 - u).$$

Для квадратичного расширения Karatsuba заменяет четыре умножения тремя:

$$\begin{aligned} t_0 &= a_0 b_0, \\ t_1 &= a_1 b_1, \\ t_2 &= (a_0 + a_1)(b_0 + b_1), \\ c_0 &= t_0 + 13t_1, \\ c_1 &= t_2 - t_0 - t_1. \end{aligned}$$

Та же идея применяется поверх $\mathbb{F}_{q^2}$ для $\mathbb{F}_{q^4}$.

Умножения на нерезиденты реализованы отдельными формулами. Например:

$$(x_0 + x_1 u)u = 13x_1 + x_0 u.$$

Это значительно дешевле общего умножения.

Таблица 3: Стоимость операций расширений MNT4-753

Операция	Gas/op
$\mathbb{F}_q$ mul	2,959
$\mathbb{F}_q$ square	2,947
$\mathbb{F}_{q^2}$ mul	11,877
$\mathbb{F}_{q^2}$ square	10,592
$\mathbb{F}_{q^4}$ mul	42,764
$\mathbb{F}_{q^4}$ square	36,606

3.5 Подготовленные разреженные линии

После подстановки точки линия Миллера имеет специальную структуру. Ее не нужно представлять как произвольный плотный элемент $\mathbb{F}_{q^4}$. В проекте реализованы:

- prepared fixed- Q cache;

Таблица 4: Эффект специализированных умножений на нерезиденты

Операция	Общий путь	Специализированный путь	Выигрыш
умножение на 13 в $\mathbb{F}_q$	2,961	1,481	2.0×
умножение на u в $\mathbb{F}_{q^2}$	12,587	1,731	7.3×
умножение на v в $\mathbb{F}_{q^4}$	42,253	1,859	22.7×

- sparse-формат коэффициентов;
- специализированное умножение аккумулятора на линию;
- pointer/packed memory API;
- потоковое чтение code-shards через EXTCODECOPY;
- экспериментальное слияние $f \leftarrow f^2 \ell(P)$ без лишнего промежуточного объекта.

Последний агрессивный fused-вариант корректен, но дает лишь около 20 тыс. gas экономии на полном single-pairing пути. Это показывает, что после оптимизации памяти доминирует уже не копирование данных, а собственно арифметика расширений.

Формальная идея разреженного умножения. Пусть аккумулятор цикла Миллера имеет общий вид:

$$f = f_0 + f_1 v, \quad f_0, f_1 \in \mathbb{F}_{q^2}.$$

После подстановки G_1 -точки подготовленная линия не является произвольным элементом $\mathbb{F}_{q^4}$: часть ее коэффициентов равна нулю или выражается через заранее известные координаты. Условно ее можно записать как:

$$\ell(P) = \ell_0(P) + \ell_1(P)v,$$

где ℓ_0, ℓ_1 вычисляются из компактного набора prepared-коэффициентов. Вместо построения плотного объекта и вызова общего умножения

$$f \leftarrow \text{Mul}_{\mathbb{F}_{q^4}}(f, \ell(P))$$

используется специализированная формула:

$$\begin{aligned} t_0 &= f_0 \ell_0, \\ t_1 &= f_1 \ell_1, \\ t_2 &= (f_0 + f_1)(\ell_0 + \ell_1), \\ f'_0 &= t_0 + u t_1, \\ f'_1 &= t_2 - t_0 - t_1. \end{aligned}$$

Умножение на u выполняется дешевым переставлением коэффициентов. В fused hot path вычисление

$$f \leftarrow f^2 \ell(P)$$

дополнительно сливается с загрузкой коэффициентов: промежуточный плотный $\mathbb{F}_{q^4}$ -объект не материализуется.

3.6 Низкоуровневые оптимизации EVM

Алгоритмические формулы были дополнены низкоуровневой реализацией:

- фиксированный размер 3×3 limb развернут вручную, без динамических циклов и динамических массивов;
- старшая половина произведения восстанавливается через MUL/MULMOD, так как EVM не имеет MULHI;
- CIOS объединяет умножение и REDC, чтобы не сохранять полное шестисловное произведение;
- промежуточные значения по возможности удерживаются на стеке;
- pointer/packed API и scratch arena уменьшают копирование вложенных структур расширений;
- lazy reduction применяется только там, где диапазон промежуточного значения формально ограничен и отложенная редукция действительно дешевле;
- специальные операции mulBy13, mulByU, mulByV заменяют общие умножения;
- prepared-кэш может читаться потоково из data-контрактов через EXTCODECOPY, без повторной передачи большого blob;
- ветвистая и branchless-редукции реализованы рядом и измерены; branchless-вариант оставлен исследовательским, поскольку в EVM он дороже.

Эта часть закрывает вопрос не только о выборе математического алгоритма, но и о том, какие расходы остаются после ручной адаптации к стековой архитектуре EVM.

4 MNT4: от наивного Tate pairing к оптимизированному пути

4.1 Наивная точка отсчета

Для честного сравнения построена наивная модель Tate pairing. Она намеренно не использует:

- короткий Ate loop;
- prepared lines;
- sparse multiplication;
- Yul hot path;
- Frobenius-декомпозицию финальной экспоненты.

Полный неоптимизированный вызов превышает практические лимиты EVM, поэто-

му измерены корректные микроблоки и построена строгая нижняя экстраполяция:

$$\begin{aligned}\text{Miller} &= 752 \cdot 973,261 + 372 \cdot 486,819 \\ &= 912,988,940 \text{ gas}, \\ \text{FE} &= 2259 \cdot 486,899 + 1100 \cdot 486,819 \\ &= 1,635,405,741 \text{ gas}.\end{aligned}$$

Итого:

$$2,548,394,681 \text{ gas}.$$

Это нижняя оценка: она не включает построение линий, проверки входов и часть накладных расходов.

4.2 Оптимизационная лестница

Не все оптимизации можно честно представить как последовательность полных контрактов: некоторые меняют стоимость одной операции поля, другие — длину цикла Миллера, третьи — способ доставки кэша. Поэтому ниже приведены три взаимосвязанные таблицы.

Таблица 5: Интегральная лестница полного MNT4-пути

Шаг	Реализация	Gas	Снижение к предыдущему шагу
0	Наивная Tate cost model	2,548,394,681	—
1	Полный optimized fixed- Q on-chain путь	258,753,182	89.85%
2	Fixed- Q prepared sparse blob	79,588,799	69.24%
3	Fixed- Q prepared sparse code-shards	79,586,596	0.003%

Таблица 6: Локальные оптимизации арифметики

Оптимизация	Что меняется	Было	Стало
Montgomery/CIOS + Yul	F_q mul вместо generic Solidity multi-limb	18,770	2,959
Karatsuba для башни	F_{q^4} mul вместо generic расширения	486,819	42,764
Умножение на 13	общая операция заменяется сложениями	2,961	1,481
Умножение на u	$(x_0 + x_1 u)u = 13x_1 + x_0 u$	12,587	1,731
Умножение на v	перестановка F_{q^2} -коэффициентов	42,253	1,859

Для строки Ate loop прочие условия специально оставлены неизменными:

$$752 \cdot 973,261 + 372 \cdot 486,819 = 912,988,940 \text{ gas},$$

$$377 \cdot 973,261 + 124 \cdot 486,819 = 427,284,953 \text{ gas}.$$

Таблица 7: Лестница цикла Миллера, финальной экспоненты и кэша

Оптимизация	Смысл	Было	Стало
Ate вместо полного Tate loop	короткая signed-запись параметра	912,988,940	427,284,953
Sparse line multiplication	линия не материализуется как общий F_{q^4}	973,261/шаг	$\approx 137,874/\text{digit}$
Prepared fixed- Q cache	line generation переносится из on-chain	258,753,182	79,588,799
Frobenius/ w_0 FE	декомпозиция вместо binary exponentiation	1,635,405,741	$\approx 28,134,825$
Aggressive fused hot path	слияние f^2 и sparse line multiplication	51,978,344	51,949,399
Code-shards	повторный blob заменяется EXTCODECOPY	79,588,799	79,586,596

Так отдельно виден эффект сокращения параметра цикла. Для финальной экспоненты измеренный overhead optimized пути вычисляется как:

$$80,113,169 - 51,978,344 = 28,134,825 \text{ gas.}$$

Оптимизации дают более чем десятикратное снижение относительно наивного пути. Однако даже prepared-вариант остается дороже обычного лимита блока Ethereum. После применения fused hot path дальнейшие локальные улучшения дают уже доли процента: доминирует число неизбежных операций в расширенном поле.

4.3 Формальная нижняя оценка стоимости Miller core

Для двухпарного уравнения:

$$e(P, Q)e(-R, S) = 1$$

используем:

$$L = 376, \quad H = 123,$$

где L — число удвоений, H — число ненулевых addition-шагов. Минимально нужны:

- L возведений аккумулятора в квадрат;
- $2L$ применений doubling-линий;
- $2H$ применений addition-линий.

Грубая нижняя оценка через измеренные $\mathbb{F}_{q^4}$ -операции:

$$\begin{aligned} 376 \cdot 36,606 &= 13,763,856, \\ 2 \cdot 376 \cdot 42,764 &= 32,157,728, \\ 2 \cdot 123 \cdot 42,764 &= 10,499,864. \end{aligned}$$

Суммарно:

$$56,421,448 \text{ gas.}$$

Реальный код дополнительно загружает sparse-коэффициенты, вычисляет значения

линий, работает с памятью и обрабатывает signed loop. Поэтому measured Miller core порядка 87.7 млн gas согласуется с формальной моделью.

5 MNT4 direct residue verifier по ePrint 2024/640

5.1 API и граница доверия

В основном fixed-shards режиме контракт проверяет:

$$e(P, Q)e(-R, S) = 1,$$

где $Q, S \in G_2$ фиксированы при развертывании, а $P, R \in G_1$ передаются пользователем. Prepared-кэш линий хранится в runtime-коде data-контрактов. Адреса shards фиксируются в конструкторе. Пользователь не может заменить их при вызове.

Это доверенная предрегистрация fixed-cache: контракт защищен от подмены кэша во время вызова, но не доказывает заново, что развертыватель построил линии правильно. Для произвольной меняющейся точки Q нужен отдельный proof корректности кэша или прямой пересчет.

5.2 Результаты MNT4

Таблица 8: Direct residue verifier MNT4

Сценарий	Что выполняется	Gas
Miller core	цикл Миллера без финальной экспоненты	87,747,219
Full equation	Miller core + обычная FE	115,997,313
Residue equation	Miller core + c -проверка вместо полной FE	93,879,746
Fixed-shards residue equation с проверкой G_1	основной fixed-cache API	93,734,789

Вклад обычной FE:

$$115,997,313 - 87,747,219 = 28,250,094 \text{ gas.}$$

Вклад residue-проверки:

$$93,879,746 - 87,747,219 = 6,132,527 \text{ gas.}$$

Экономия:

$$22,117,567 \text{ gas.}$$

Оптимизация финальной экспоненты работает, но не устраняет главный расход: цикл Миллера.

6 Проверка цикла Миллера через полиномиальные отношения

6.1 Математическая идея

Следующий шаг статьи [1] — не исполнять все умножения расширенного поля on-chain , а проверить их как полиномиальные отношения. В статье сформулирована общая идея. Для MNT4-753 потребовалась отдельная адаптация: нужно связать quotient-check не с абстрактными умножениями, а с реальными состояниями общего multi-Miller аккумулятора, fixed- Q /fixed- S кэшем и residue-свидетельством c .

Пусть:

$$\mathbb{F}_{q^4} \simeq \mathbb{F}_q[X]/(p(X)).$$

Для умножения $a \cdot b = c$ существует частный многочлен $Q(X)$:

$$a(X)b(X) - c(X) = Q(X)p(X).$$

Verifier выбирает случайную точку α и проверяет:

$$a(\alpha)b(\alpha) - c(\alpha) = Q(\alpha)p(\alpha).$$

Для набора отношений используется challenge β :

$$\sum_i \beta^i R_i(X) = Q(X)Z_H(X).$$

Для цикла Миллера R_i кодируют реальные переходы:

$$f_{i+1} = f_i^2 \ell_{Q,i}(P) \ell_{S,i}(-R).$$

Чтобы prover не подменил значения после получения α , нужны commitments и доказательства открытия.

6.2 Построенная MNT4-модель

Для MNT4 была построена следующая модель.

Блочное сжатие цикла. Несколько раундов объединяются в один блок. Для двух шагов:

$$f_{i+2} = f_i^4 \ell_i^2 \ell_{i+1}.$$

Для блока длины d :

$$f_{i+d} = f_i^{2^d} \prod_{j=0}^{d-1} \ell_{i+j}^{2^{d-1-j}}.$$

В модели используется $d = 5$. Тогда один блочный переход для уравнения сопряжений имеет вид:

$$F_{b+1} = F_b^{32} G_b^Q(P) G_b^S(-R) c^{\kappa_b}.$$

Здесь G_b^Q и G_b^S — сжатые делители для зафиксированных точек Q, S , а κ_b — заранее известный показатель residue-рекурсии.

Проверка вычисления делителей. Для fixed- Q /fixed- S коэффициенты делителей строятся off-chain и фиксируются root-ом. Значение $G_b^Q(P)$ не должно выбираться prover-ом произвольно. Поэтому строится таблица вычисления по схеме Горнера:

$$h_{j+1} = h_j P_x + \gamma_j,$$

где γ_j — зафиксированный коэффициент сжатого делителя. Аналогично вычисляется $G_b^S(-R)$. Локальные отношения Горнера и блочные переходы собираются в quotient polynomial.

Что доказано для модели. Для спецификации были формализованы:

- связь пяти пошаговых переходов Миллера с одним сжатым делителем;
- корректность Horner-рекурсии для fixed-divisor коэффициентов;
- quotient identity для локальных переходов;
- Merkle-binding динамических таблиц;
- Fiat–Shamir transcript и детерминированный порядок запросов;
- ordinary-FRI low-degree проверка;
- итоговая soundness-оценка как сумма ошибок FRI, Schwartz–Zippel, Merkle-binding и Fiat–Shamir модели.

Таким образом, исследован не эвристический “кэш строк”, а конкретная неинтерактивная схема проверки реальных переходов цикла Миллера.

6.3 KZG over BN254

KZG дает короткое доказательство открытия и дешевую on-chain проверку благодаря BN254 precompile. В проекте реализован реальный opening layer:

$$C_j = \text{Commit}(w_j), \quad C_Q = \text{Commit}(Q).$$

Для одного полинома проверяется стандартное отношение:

$$e(C - yG_1 + x\pi, G_2) = e(\pi, \tau G_2).$$

Здесь C — commitment, x — точка открытия, $y = f(x)$ — заявленное значение, π — opening proof, τG_2 — элемент SRS. Measured стоимость одной on-chain opening-проверки составляет около:

$$133,039 \text{ gas}.$$

Проблема переносится внутрь circuit: MNT4-арифметика становится non-native относительно BN254.

Таблица 9: Оценка BN254 non-native constraints

Операция	Constraints
MNT4 $\mathbb{F}_q$ mul как non-native	3,692
MNT4 $\mathbb{F}_{q^2}$ mul как non-native	11,300
Модель MNT4 $\mathbb{F}_{q^4}$ mul как non-native	63,436
Приближенная sparse Miller relation	63,309,128

Число 63,309,128 получается не из размера KZG proof. Оно относится к relation, которую нужно доказать до открытия: каждое MNT4 $\mathbb{F}_{q^4}$ умножение при переносе в BN254 R1CS раскладывается на non-native операции. В проекте отдельно измерены 3,692 constraints для одного MNT4 $\mathbb{F}_q$ -умножения через emulated field, 11,300 для $\mathbb{F}_{q^2}$ и 63,436 для модели $\mathbb{F}_{q^4}$. Подстановка числа sparse Miller переходов дает ориентир 63.3 млн constraints. То есть KZG уменьшает gas verifier-a, но возвращает исходную проблему роста off-chain constraints.

6.4 Merkle/FRI

В альтернативном направлении prover коммитится к таблицам через Merkle root. Verifier проверяет Merkle-пути открытых строк. FRI (*Fast Reed–Solomon Interactive Oracle Proof of Proximity*) доказывает, что открываемые таблицы согласуются с многочленами ограниченной степени. Fiat–Shamir-преобразование делает схему неинтерактивной: challenges детерминированно вычисляются из transcript.

В проекте реализованы:

- Merkle opening layer;
- исполняемый архивный DEEP-FRI отрицательный эксперимент;
- спецификация блочно-сжатого ordinary-FRI пути;
- актуальная ordinary-FRI cost model для нативного MNT4-поля.

Полноценная production-замена Miller loop не заявляется: результат используется как строгая stop/go модель стоимости.

Для MNT4 один элемент поля занимает:

$$3 \cdot 32 = 96 \text{ байт.}$$

Умеренная модель открытий:

$$4096 \cdot 96 + 128 \cdot 16 \cdot 32 + 16 \cdot 32 = 459,264 \text{ байт.}$$

Актуальная ordinary-FRI модель для 128-битного профиля:

Из чего складывается нижняя оценка. Для каждого случайного запроса verifier должен получить раскрытые значения столбцов, соседние значения для локального перехода, Merkle-path и элементы слоев FRI. Поскольку один MNT4-элемент занимает 96

Таблица 10: Ordinary-FRI cost model

Показатель	Значение
Число запросов	576
Оценка soundness	135.76 бит
FRI schedule	[2, 2, 2, 4]
Размер доказательства	2,072,224 байт
Строгая нижняя оценка	51,359,352 gas
Ожидаемая модельная оценка	78,340,624 gas

байт, даже до исполнения арифметики calldata становится существенным компонентом стоимости. Актуальная модель суммирует:

$$\text{gas}_{\min} = \text{gas}_{\text{calldata}} + \text{gas}_{\text{Merkle}} + \text{gas}_{\text{FRI folding}} + \text{gas}_{\text{local checks}}.$$

Для профиля с 576 запросами строгая нижняя оценка равна 51,359,352 gas. Она учитывает минимально необходимые данные и операции, но не претендует на стоимость готового Solidity-контракта. Исполняемая модель с реалистичными накладными расходами дает:

$$78,340,624 \text{ gas}.$$

Почему не выбран DEEP-FRI production-путь. Более консервативная DEEP-FRI микротрасса была реализована как отрицательный эксперимент. Она проверяет более широкую временную трассу и требует большего числа раскрытий. После профилирования этот путь перенесен в `research_variants`: он криптографически содержателен, но не дает практического gas-выигрыша.

Вывод: Merkle/ordinary-FRI может приблизиться к Article640 fixed-shards verifier и в оптимистичной нижней модели пересечь уровень 60 млн gas. Однако реалистичная оценка остается около 78.3 млн gas. Стоимость переносится из прямых F_{q^4} -умножений в calldata, Merkle hashing, раскрытия таблиц и FRI folding. Поэтому направление подробно исследовано, но не принято как финальная production-замена цикла Миллера.

7 MNT6-753 и цикл MNT4/MNT6

7.1 Почему нужна вторая кривая

MNT4-753 и MNT6-753 образуют цикл полей:

$$\mathbb{F}_r(\text{MNT4}) = \mathbb{F}_q(\text{MNT6}),$$

$$\mathbb{F}_r(\text{MNT6}) = \mathbb{F}_q(\text{MNT4}).$$

Для будущего folding это принципиально: арифметику одной стороны можно выражать ближе к родному полю другой стороны, избегая наиболее грубой BN254 non-native эмуляции.

7.2 MNT6 arithmetic и residue verifier

Для MNT6 реализована башня:

$$\mathbb{F}_q \longrightarrow \mathbb{F}_{q^3} \longrightarrow \mathbb{F}_{q^6}.$$

Реализованы packed/pointer API, scratch arena, prepared cache, потоковое чтение code-shards, fused line multiplication и Frobenius/w0 FE.

Таблица 11: Базовые операции MNT6

Операция	Gas/op
$\mathbb{F}_q$ mul	2,314
$\mathbb{F}_q$ square	2,301
$\mathbb{F}_{q^3}$ mul	36,472
$\mathbb{F}_{q^3}$ square	26,813
$\mathbb{F}_{q^6}$ mul	166,774
$\mathbb{F}_{q^6}$ square	113,788

После финального аудита MNT6 verifier приведен к той же архитектуре, что и MNT4: используется общий multi-Miller аккумулятор и один квадрат на раунд. Из $r_{\text{MNT6}} = q - N$ следует:

$$F \cdot c^{N-q} = F \cdot c^{-r} = 1.$$

Лемма 1. Корректность башни расширений MNT6. Пусть:

$$\mathbb{F}_{q^3} = \mathbb{F}_q[v]/(v^3 - 11), \quad \mathbb{F}_{q^6} = \mathbb{F}_{q^3}[w]/(w^2 - v).$$

При выборе нерезидентов из параметров MNT6-753 эти многочлены неприводимы, поэтому конструкции задают поля степеней 3 и 6. Следовательно, Karatsuba-формулы, отображения Фробениуса и обращение элементов реализуются в том же поле, в котором лежит результат MNT6-сопряжения. Константы и reference-значения кросс-проверены с arkworks.

Лемма 2. Общий аккумулятор для уравнения сопряжений. Пусть на раунде i индивидуальные аккумуляторы обновляются как:

$$f'_Q = f_Q^2 \ell_{Q,i}(P), \quad f'_S = f_S^2 \ell_{S,i}(-R).$$

Для произведения $F = f_Q f_S$ имеем:

$$F' = f'_Q f'_S = (f_Q f_S)^2 \ell_{Q,i}(P) \ell_{S,i}(-R).$$

Следовательно, вместо двух возведений аккумуляторов в квадрат достаточно одного. Именно эта формула используется в MNT6 fixed-shards verifier.

Теорема 1. Residue-отношение для MNT6. Для параметров MNT6:

$$r_{\text{MNT6}} = q_{\text{MNT6}} - N.$$

Если результат общего цикла Миллера равен F и prover предоставляет c такое, что $F = c^r$, то:

$$F \cdot c^{-r} = F \cdot c^{N-q} = 1.$$

Возведение в степень q реализуется отображением Фробениуса. Поэтому контракт проверяет то же математическое условие, что и полная финальная экспонента для уравнения сопряжений, но не выполняет полный exponentiation chain.

Таблица 12: Результаты MNT6

Режим	Gas
Miller loop, packed pointer blob	93,254,054
Packed Frobenius/w0 FE	38,428,108
Полное single pairing: Miller + packed FE	131,685,843
Диагностический residue digest одного pairing	103,277,505
Два Miller loop + полная FE для equation	226,078,963
Общий multi-Miller + c -проверка для equation	172,004,717

Экономия общего residue verifier относительно контрольного equation baseline:

$$226,078,963 - 172,004,717 = 54,074,246 \text{ gas},$$

то есть 23.92%.

7.3 Cycle-native accounting

Rust-модуль проверяет параметры цикла через arkworks, строит reference pairings и считает operation/constraints accounting для будущего native relation layer.

Теорема 2. Связь полей цикла. Для выбранных параметров:

$$\mathbb{F}_r(\text{MNT4}) = \mathbb{F}_q(\text{MNT6}), \quad \mathbb{F}_r(\text{MNT6}) = \mathbb{F}_q(\text{MNT4}).$$

Равенства проверяются программно сравнением модулей arkworks. Поэтому базовые умножения одной стороны цикла могут выражаться как нативные multiplication constraints в скалярном поле другой стороны, а не как 753-битная арифметика, эмулируемая поверх BN254.

Для сравнения:

$$\text{BN254 non-native sparse Miller estimate} = 63,309,128,$$

$$\text{Sonobe/CycleFold-like anchor} \approx 9,000,000.$$

Число 73,068 нельзя называть стоимостью готового CycleFold. Это ручная модель multiplication constraints для подготовленных relation fragments. Она показывает поря-

Таблица 13: Предварительная модель MNT-cycle-native relation

Сторона	Башня	Miller rounds	Additions	Prepared relation
MNT4	$\mathbb{F}_{q^2}/\mathbb{F}_{q^4}$	376	124	24,126
MNT6	$\mathbb{F}_{q^3}/\mathbb{F}_{q^6}$	376	123	48,942
Сумма строительных блоков				73,068

док потенциального выигрыша нативной постановки и обосновывает направление следующего этапа.

Оценка будущего cycle-native folding слоя. Внутреннее ядро relation уже посчитано по реальным формулам проекта:

$$24,126 + 48,942 = 73,068$$

нативных multiplication constraints. Для MNT4 ручная модель дополнительно сверялась с собранным R1CS-фрагментом: расхождение составляет единицы ограничений. Полный CycleFold-circuit еще не реализован, поэтому нельзя выдавать одно точное итоговое число. Можно честно построить чувствительность к накладным расходам folding-обвязки:

Таблица 14: Проекция cycle-native constraints при разных накладных расходах

Сценарий	Коэффициент к relation core	Constraints
Только подготовленные fragments	1×	73,068
Легкая folding-обвязка	2×	146,136
Умеренная folding-обвязка	4×	292,272
Консервативная folding-обвязка	8×	584,544
Верхняя исследовательская граница	12×	876,816

Эта таблица не является замером готового CycleFold. Она отвечает на вопрос, какой запас допускает измеренное ядро до уровня Sonobe/CycleFold-like ориентира около 9 млн constraints. Даже при двенадцатикратной обвязке оценка остается меньше 1 млн. Следовательно, полученные результаты поддерживают основной тезис работы: MNT4/MNT6 цикл потенциально позволяет кратно снизить constraints, если следующий этап реализовать нативно, а не переносить MNT-арифметику в BN254.

Если механически сложить лучшие прямые EVM equation verifier-ы двух сторон, получится:

$$93,734,789 + 172,004,717 = 265,739,506 \text{ gas.}$$

Эта сумма иллюстрирует границу прямого on-chain исполнения. Будущий folding не должен выполнять оба verifier-а on-chain; его смысл как раз в переносе relation layer в нативный цикл.

8 Исследовательское направление lollipop-305

8.1 Идея

Статья *Cycles of supersingular elliptic curves for pairing-based proof systems* [2] предлагает конструкции, в которых часть арифметики выполняется над меньшими полями. Для исследовательского примера lollipop-305 выбран параметр:

$$x = 8004046504391788107635887004283725454478544674,$$

$$p = x^2 - x + 1, \quad q = x^2 + 1.$$

Оба основных поля имеют размер около 305 бит и помещаются в два слова EVM. Цель этого направления — проверить, является ли уменьшение числа limb-ов достаточным условием практического выигрыша. Это не production-предложение: параметры lollipop-305 выбраны как исследовательский прототип для измерения архитектурного эффекта.

8.2 Кривые конструкции

Таблица 15: Pairing-компоненты lollipop-305

Часть	Кривая и поле	Целевое поле
stick	pairing-friendly prototype над $\mathbb{F}_p$	$\mathbb{F}_{p^4}$
cycle E	$E/\mathbb{F}_{p^2} : y^2 = x^3 + (\mu + 1)x$	$\mathbb{F}_{p^4}$
cycle $\hat{E}$	$\hat{E}/\mathbb{F}_{q^2} : y^2 = x^3 + (\eta + 2)$	$\mathbb{F}_{q^6}$

Полная конструкция содержит три pairing-компонента:

1. “палочку” — pairing-friendly prototype над $\mathbb{F}_p$;
2. первую supersingular cycle-кривую $E/\mathbb{F}_{p^2}$;
3. вторую supersingular cycle-кривую $\hat{E}/\mathbb{F}_{q^2}$.

Дополнительно статья использует меньшую непаринговую кривую для рекурсивного сценария, но она не является отдельным EVM pairing verifier.

Проверены отношения:

$$\#E(\mathbb{F}_{p^2}) = p^2 + 1 = qN_q,$$

$$\#\hat{E}(\mathbb{F}_{q^2}) = q^2 - q + 1 = pN_p.$$

Лемма 3. Исправление башни для stick/ E -части. Первоначальный кандидат:

$$\mathbb{F}_{p^4} = \mathbb{F}_{p^2}[v]/(v^2 - u), \quad u^2 = -1,$$

не задает нужное поле: для выбранного p элемент u является квадратом в $\mathbb{F}_{p^2}$. В работе вычислительно найден и проверен неквадрат:

$$\xi = 1 + u.$$

Поэтому используется исправленная башня:

$$\mathbb{F}_{p^4} = \mathbb{F}_{p^2}[v]/(v^2 - \xi).$$

Лемма 4. Twist/Frobenius-отношение. Для исправленной башни построено отображение из twist в исходную кривую:

$$\psi(X, Y) = (\xi X, \xi v Y).$$

Rust backend проверяет:

$$[r]P = \mathcal{O}, \quad [r]\psi(Q') = \mathcal{O}, \quad \pi_p(\psi(Q')) = [p]\psi(Q'),$$

а также нетривиальность результата сопряжения и условие принадлежности целевой подгруппе.

Лемма 5. Корректная форма для $\hat{E}$. Для третьей кривой выбрана башня:

$$\mathbb{F}_{q^6} = \mathbb{F}_{q^2}[w]/(w^3 - \rho),$$

где:

$$\rho^2 = \frac{2 + \eta}{2 - \eta}.$$

Distortion/Frobenius map имеет вид:

$$\psi_{\hat{E}}(x, y) = (w^2 x^q, \rho y^q).$$

Для $\hat{E}$ упрощенная Tate-style формула не проходит. Корректным reference путем является Weil-отношение:

$$e_p(P, Q) = (-1)^p \frac{f_{p,P}(Q)}{f_{p,Q}(P)}.$$

Отсюда следует verifier relation:

$$f_{p,P}(Q)f_{p,-P}(Q) = f_{p,Q}(P)f_{p,Q}(-P).$$

8.3 Оптимизированная 2-limb арифметика

Для двухсловного Montgomery-умножения generic lower bound содержит:

$$4 \text{ произведения для } ab + 4 \text{ произведения для REDC.}$$

Для lollipop-305 старший limb модуля меньше 2^{49} . Поэтому:

$$a_1 b_1 < 2^{98} < 2^{256}.$$

Одно полное 512-битное восстановление можно заменить обычным MUL. Дополнительно убрана запись limb-а, который сразу удаляется CIOS-сдвигом.

Эта часть не является упрощенным демонстрационным скриптом. Для 305-битных полей проделан тот же класс инженерной работы, что и для 753-битной арифметики MNT4:

- Montgomery/CIOS-представление;
- ручной Yul hot path;
- специальная обработка малого старшего limb-a;
- арифметика расширений $\mathbb{F}_{p^2}$, $\mathbb{F}_{p^4}$, $\mathbb{F}_{q^2}$ и $\mathbb{F}_{q^6}$;
- Karatsuba и специализированные умножения на нерезиденты;
- prepared lines, residue-пути и fixed-cache режимы;
- Rust cross-check и gas-бенчмарки.

Таблица 16: Сравнение 2-limb lollirop-305 и 3-limb MNT4

Операция	lollirop-305	MNT4-753	Выигрыш
F_p mul	1,018	2,959	$2.91\times$
F_p square	940	2,947	$3.14\times$
F_{p^2} mul	4,171	11,877	$2.85\times$
F_{p^2} square	3,254	10,592	$3.26\times$
F_{p^4} mul	14,942	42,764	$2.86\times$
F_{p^4} square	13,211	36,606	$2.77\times$

8.4 Почему для третьей кривой нужны два аккумулятора

Для stick и E можно использовать Article640-style residue verifier. Для $\widehat{E}/\mathbb{F}_{q^2}$ степень вложения равна 3. Стандартное удаление знаменателей линий не работает в требуемой форме. Поэтому backend и Solidity отдельно накапливают:

$$F_{\text{num}}, \quad F_{\text{den}}.$$

На doubling-шаге:

$$\begin{aligned} F_{\text{num}} &\leftarrow F_{\text{num}}^2 \ell_{\text{dbl}}(P), \\ F_{\text{den}} &\leftarrow F_{\text{den}}^2 v_{\text{dbl}}(P). \end{aligned}$$

На addition-шаге:

$$\begin{aligned} F_{\text{num}} &\leftarrow F_{\text{num}} \ell_{\text{add}}(P), \\ F_{\text{den}} &\leftarrow F_{\text{den}} v_{\text{add}}(P). \end{aligned}$$

В конце проверяется отношение:

$$c^p F_{\text{den}} = F_{\text{num}}.$$

Это эквивалентно residue-проверке частного:

$$F = F_{\text{num}}/F_{\text{den}}.$$

Ведение двух аккумуляторов и 305-битное возведение c^p в $\mathbb{F}_{q^6}$ делают третью часть заметно дороже.

8.5 Результаты lollipop-305

Таблица 17: Pairing-verifier-ы lollipop-305

Часть	Режим	Function gas
stick	residue FE	8,671,771
cycle $E/\mathbb{F}_{p^2}$	residue FE	18,283,781
cycle $\hat{E}/\mathbb{F}_{q^2}$	Weil fallback	200,977,272
cycle $\hat{E}/\mathbb{F}_{q^2}$	prepared-Ate raw $F_{\text{num}}/F_{\text{den}}$	76,422,749
cycle $\hat{E}/\mathbb{F}_{q^2}$	prepared-Ate + residue	106,441,661
Полный lollipop-cycle	stick + E + улучшенный $\hat{E}$	133,397,213

Для fixed-cache режима дополнительно сравнивались calldata+commitment и code-shards:

135,916,985 gas против 133,651,154 gas.

Code-shards экономят около 2.27 млн gas за счет исключения повторной передачи примерно 273 КБ подготовленных линий.

Lollipop-305 показывает важный положительный факт: уменьшение числа limb-ов действительно дает почти трехкратный выигрыш на базовой арифметике. Но этого недостаточно для автоматического выигрыша полного цикла: структура pairing, степень расширения и возможность удаления знаменателей не менее важны, чем размер поля.

9 Сводные результаты и trade-off

Таблица 18: Основные результаты работы

Подход	Что измеряется	Результат
Наивный Tate baseline	строгая нижняя экстраполяция	2.55 млрд gas
MNT4 full optimized on-chain	fixed- Q , линии строятся on-chain	258.8 млн gas
MNT4 prepared sparse	кэш линий подготовлен заранее	79.6 млн gas
MNT4 Article640 residue	fixed-shards equation с проверкой G_1	93.7 млн gas
MNT6 full equation	два Miller loop + packed FE	226.1 млн gas
MNT6 Article640 residue	общий multi-Miller accumulator	172.0 млн gas
KZG opening BN254	короткая on-chain opening-проверка	133 тыс. gas

Подход	Что измеряется	Результат
KZG + MNT4 relation	BN254 non-native sparse Miller estimate	63.3 млн constraints
Merkle/ordinary-FRI	ожидаемая модельная стоимость	78.3 млн gas
Merkle/ordinary-FRI	размер доказательства	2.07 МБ
MNT-cycle-native relation	MNT4 + MNT6 fragments	73,068 операций
lollipop-305	полный улучшенный исследовательский цикл	133.4 млн gas

9.1 Три формы стоимости

Полученные результаты показывают не один, а три взаимосвязанных вида расходов.

On-chain arithmetic. Если считать MNT-сопряжение непосредственно в EVM, придется платить gas за трехсловную арифметику, расширения поля и сотни шагов цикла Миллера.

Off-chain constraints. Если перенести вычисление в proof над BN254, MNT-арифметика становится non-native. Gas on-chain уменьшается, но растет число constraints, размер witness и время работы prover-a.

Calldata и proof size. Если использовать нативные таблицы и Merkle/FRI, можно уменьшить зависимость от BN254 non-native arithmetic, но стоимость возвращается через Merkle-пути, раскрытия таблиц и calldata.

9.2 Экономическая интерпретация

Для иллюстрации использован снимок на **1 июня 2026 года, 13:51 MSK**. На момент снимка Etherscan показывал:

ETH price ≈ 1982.85 USD,

Ethereum standard gas price ≈ 0.294 gwei.

RPC Arbitrum One возвращал:

Arbitrum One execution gas price ≈ 0.020 gwei.

Источники снимка: Etherscan Gas Tracker [4] и публичный RPC Arbitrum One [5]. Для Ethereum mainnet:

$$\text{cost}_{USD} = \text{gas} \cdot \text{gas price}_{\text{gwei}} \cdot 10^{-9} \cdot 1982.85.$$

Таблица 19: Стоимость эквивалентного EVM-исполнения по снимку 01.06.2026

Режим	Gas	Ethereum mainnet	Arbitrum L2 execution
MNT4 prepared sparse	79,586,596	\$46.40	\$3.16
MNT4 Article640 residue	93,734,789	\$54.64	\$3.72
MNT6 Article640 residue	172,004,717	\$100.27	\$6.82
lollipop full research-cycle	133,397,213	\$77.76	\$5.29
Merkle/FRI lower bound	51,359,352	\$29.94	\$2.04
Merkle/FRI expected model	78,340,624	\$45.67	\$3.11

Столбец Arbitrum показывает только стоимость L2-исполнения одинакового числа EVM gas. Реальная комиссия Arbitrum дополнительно включает стоимость публикации данных в родительскую сеть. Согласно документации Arbitrum, пользователь платит за L2 execution и за L1 data posting [6]. Для prepared blob и Merkle/FRI пути эта оговорка принципиальна: большой calldata увеличивает именно L1 data fee. Кроме того, очень крупный монолитный вызов может потребовать разбиения на несколько транзакций даже в L2.

Таким образом, L2 снижает денежную цену арифметики, но не устраняет архитектурное ограничение. Варианты с большим calldata остаются дорогими по публикации данных, а тяжелые on-chain варианты требуют много ресурсов исполнения и не всегда помещаются в один вызов.

10 Итоговые выводы

10.1 Заккрытие замечаний научного руководителя

Таблица 20: Соответствие выполненной работы замечаниям руководителя

Замечание	Что выполнено
Сравнить алгоритмы умножения и редукции	Реализованы и измерены Montgomery/CIOS, Barrett, FIOS, Comba/SOS, square-Comba, lazy reduction и branchless reduction. Для башен расширений исследованы Karatsuba и специальные умножения на нерезиденты.
Обосновать оптимальность реализации	Построена opcode-only нижняя граница 3-limb умножения, реализован ручной all-stack Yul hot path, исследованы pointer/packed API, scratch arena, fused sparse multiplication и потоковое чтение code-shards. После fused оптимизации локальный резерв улучшения составляет доли процента полного Miller path.

Замечание	Что выполнено
Дать оценки сложности	Оценки построены на нескольких уровнях: EVM opcode, F_q , F_{q^2}/F_{q^4} и F_{q^3}/F_{q^6} операции, число Miller rounds, стоимость FE, calldata, Merkle paths и constraints. Каждая интегральная цифра сопровождается указанием: измерение, исполняемая модель или ручная оценка.
Исследовать более совершенные техники	Реализованы shared multi-Miller accumulator, prepared sparse lines, residue-проверка FE, KZG-opening, Merkle/DEEP-FRI отрицательный эксперимент, ordinary-FRI cost model, MNT4/MNT6 cycle-native accounting и lollipop-305 по ePrint 2024/1627.
Провести всестороннее исследование	Сравнены три фундаментальные формы стоимости: on-chain arithmetic, off-chain constraints и calldata/proof size. Получен граничный результат: локальные Solidity-оптимизации существенно помогают, но не устраняют системный trade-off без нового proof/folding слоя.

1. Реализована и измерена оптимизированная 753-битная арифметика MNT4-753/MNT6-753 в Solidity/Yul. Для MNT4 экспериментально обоснован выбор Montgomery/CIOS, Karatsuba, специализированных умножений на нерезиденты, sparse lines и packed hot path.
2. Построена оптимизационная лестница от наивного Tate baseline (2.55 млрд gas по строгой нижней экстраполяции) до prepared MNT4 пути (79.6 млн gas). Это показывает эффект оптимизаций и одновременно сохраняющуюся границу EVM.
3. Реализована идея residue-проверки по ePrint 2024/640. Для MNT4 финальная часть сокращается с 28.25 млн до 6.13 млн gas overhead. Для MNT6 общий multi-Miller residue verifier снижает equation baseline на 54.07 млн gas.
4. Исследована замена цикла Миллера полиномиальной проверкой. KZG уменьшает on-chain gas, но переносит стоимость в BN254 non-native constraints. Merkle/FRI избегает части этого переноса, но требует большого proof и calldata.
5. Реализован предварительный MNT4/MNT6 cycle-native слой. Его accounting дает 24,126 и 48,942 multiplication operations для подготовленных fragments. Это не готовый CycleFold, но обоснованный строительный блок будущего folding.
6. Исследована конструкция lollipop-305. Двухсловная арифметика действительно дешевле MNT4 примерно в 2.8–3.3 раза. Однако третья кривая требует двух аккумуляторов числителя и знаменателя, поэтому полный прямой EVM-цикл остается дорогим.

Главный вывод работы:

Без MNT-prescompile или нового proof-system слоя нельзя одновременно получить малую on-chain стоимость и малую off-chain стоимость простым переносом вычислений. Выполненная работа локализует причины этого ограничения, подтверждает их кодом и измерениями и показывает наиболее перспективное направление продолжения: MNT4/MNT6 cycle-native relation layer с последующим folding.

11 Воспроизводимость

Финальный очищенный проект расположен в каталоге:

```
/Users/a.i.semenov/diploma-final.
```

Основной запуск:

```
cd /Users/a.i.semenov/diploma-final  
./scripts/run_all.sh
```

Он выполняет gas-бенчмарки арифметики, MNT4/MNT6 Article640-модули, Rust fixture cross-check, MNT-cycle accounting и ordinary-FRI cost model.

Полный аудит завершен успешно:

- MNT6 Article640: 13/13 Foundry-тестов;
- Rust fixture MNT6 побайтно совпадает с tracked JSON;
- MNT-cycle accounting: 5/5 тестов;
- ordinary-FRI cost model воспроизводится;
- основной MNT6 verifier имеет runtime-размер 20,312 байт, что укладывается в EIP-170.

Список литературы

- [1] Andrija Novakovic, Liam Eagen. *On Proving Pairings*. Cryptology ePrint Archive, Paper 2024/640, 2024. URL: <https://eprint.iacr.org/2024/640.pdf>.
- [2] Craig Costello, Pratyush Korpai. *Cycles of supersingular elliptic curves for pairing-based proof systems*. Cryptology ePrint Archive, Paper 2024/1627, 2024. URL: <https://eprint.iacr.org/2024/1627.pdf>.
- [3] Privacy and Scaling Explorations. *Sonobe: a modular library for folding schemes*. URL: <https://github.com/privacy-ethereum/sonobe>.
- [4] Etherscan. *Ethereum Gas Tracker*. URL: <https://etherscan.io/gastracker>. Снимок данных: 01.06.2026, 13:51 MSK.
- [5] Offchain Labs. *Arbitrum One public RPC endpoint*. URL: <https://arb1.arbitrum.io/rpc>. Снимок данных: 01.06.2026, 13:51 MSK.

[6] Offchain Labs. *Arbitrum Nitro: gas and fees*. URL: <https://docs.arbitrum.io/how-arbitrum-works/gas-fees>.